

CSRMS — Campus Service Request Management System

NADV 744 Advanced Development Systems · Group Assignment 2026

Sol Plaatje University

August 2026

- [Cover details](#)
- [1. Background and context](#)
 - [1.1 The problem](#)
 - [1.2 Why it matters in ICT](#)
 - [1.3 Aim and objectives](#)
 - [1.4 Plan and resources](#)
- [2. System design](#)
 - [2.1 Architecture](#)
 - [2.2 Data flow](#)
 - [2.3 Data model](#)
 - [2.4 Request lifecycle](#)
 - [2.5 API endpoints](#)
 - [2.6 Security design](#)
 - [2.7 IoT auto-request rules](#)
- [3. Implementation](#)
 - [3.1 Stack and layout](#)
 - [3.2 Engineering decisions worth defending](#)
 - [3.3 Version control practice](#)
- [4. Testing and evaluation](#)
 - [4.1 Test plan and unit tests](#)
 - [4.2 Performance observations](#)
 - [4.3 Evidence](#)
- [5. Results and discussion](#)
- [6. Limitations and future improvements](#)
- [7. Conclusion](#)
- [Appendix A — Running the system](#)

Cover details

Module	NADV 744 — Advanced Development Systems
Assessment	Group Assignment 2026 (100 marks)

Institution	Sol Plaatje University, Northern Cape
Due date	24 August 2026

Team members:

Full name	Student number
Karabelo Nthoroane	202328762
Khulani Hlebeya	202302091
Kegomoditswe Mongale	201718863
Metswi kabo	202104098

1. Background and context

1.1 The problem

Campus maintenance at Sol Plaatje University currently runs on hallway conversations, WhatsApp messages and paper slips. A student who notices a broken window has no reliable way to report it; at best they mention it to a lecturer and hope the message travels. When reports do reach facilities staff, they arrive without structure: no category, no location standard, no record of who is handling the job or whether it was ever finished. The same geyser can be reported three times by three different people while a fourth leak goes unnoticed because nobody happens to walk past it.

The consequences are familiar to everyone on campus. IT support hears about network outages from students on social media before it hears about them through any official channel. Maintenance backlogs are invisible until something fails completely. Nobody can answer the basic management question: *how many problems were reported this month, how long did they take to fix, and where are the worst buildings?*

There is also a class of problem that should never depend on a human noticing anything. A leaking pipe does not wait for business hours. A server room that starts overheating at 2am cannot be reported by a student who is not there. These failures need sensors that report automatically, not a passer-by.

1.2 Why it matters in ICT

This project sits squarely in the ICT service-management tradition. It applies, in miniature, the same ideas that run production IT organisations:

- **Service-oriented architecture.** Every capability — accounts, requests, telemetry, dashboards — is exposed as a REST API speaking JSON, so any client (web app, mobile app, sensor) consumes the same contract.
- **IoT/telemetry pipelines.** Simulated ESP32 sensors post readings to an ingest API that authenticates devices, stores measurements and applies threshold rules — the same shape as an industrial monitoring pipeline.
- **Workflow automation.** Requests move through an enforced state machine (PENDING → ASSIGNED → IN_PROGRESS → RESOLVED) with a full audit trail, mirroring ITIL-style incident management.

- **Security engineering.** JWT authentication, role-based access control down to individual objects, and separate hashed credentials for devices are all first-class requirements rather than afterthoughts.

The system is deliberately small enough to build in a semester but exercises every layer a real deployment would: database, API, web frontend, embedded clients, automated tests and version control.

1.3 Aim and objectives

Aim: deliver a working campus service request system in which students log problems, staff resolve them through a managed workflow, administrators see the whole picture, and simulated IoT sensors raise requests on their own when physical conditions demand it.

Objectives:

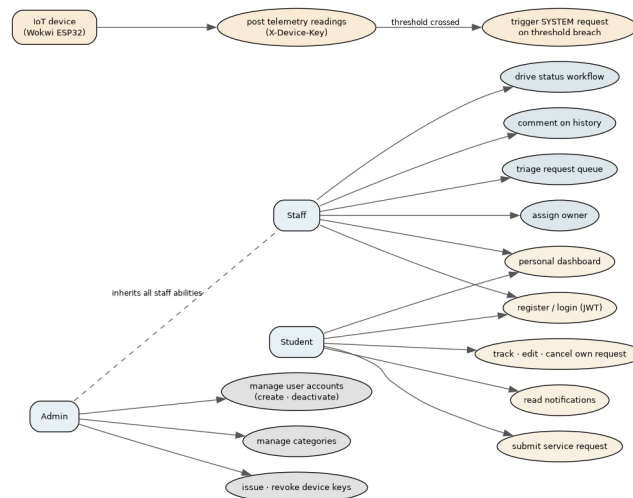
1. A REST API covering registration, JWT authentication, role-based authorisation, request lifecycle management with history, notifications, dashboards and telemetry ingest.
2. A web application usable by all three roles, with charts and live sensor views.
3. Three Wokwi-simulated ESP32 devices that post readings and trigger automatic requests under defined thresholds.
4. An automated test suite proving both functionality and security properties.
5. Documentation sufficient for a stranger to run the whole system.

1.4 Plan and resources

The group worked from a shared monorepo on a single `main` branch, committing in small units prefixed `feat:` / `fix:` / `test:` / `docs:` so the history reads as a narrative of the build. Work proceeded backend-first (the API is the contract everything else depends on), then the frontend against a seeded database, then the device simulations, with tests written alongside each component rather than bolted on at the end.

Resources: Python/Django REST Framework for the API, SQLite for development (MySQL-ready for deployment), React with Vite and Tailwind CSS for the web client, Wokwi's virtual ESP32 for hardware simulation, Postman for manual API exploration, Graphviz for diagrams, and GitHub for version control. No physical hardware was required.

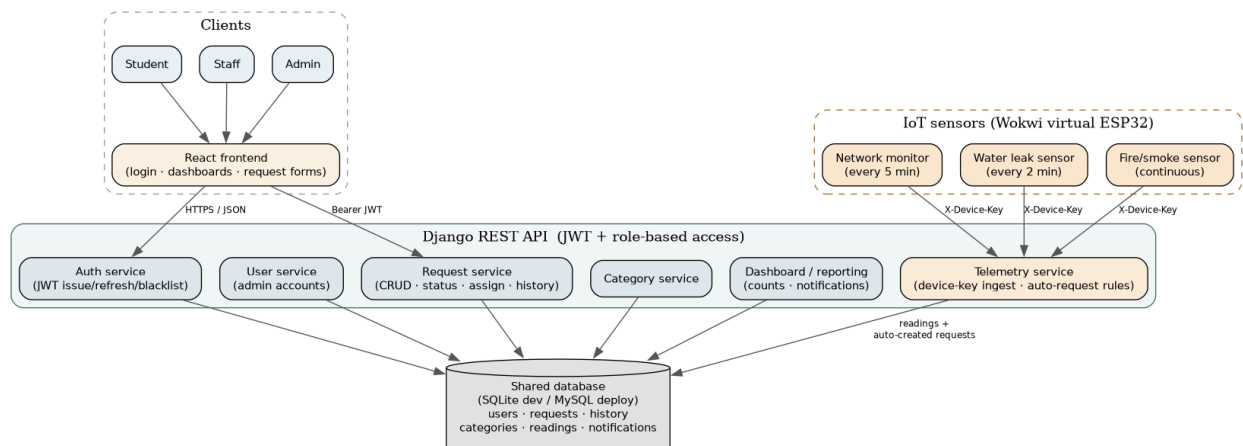
2. System design



Use cases by Student, Staff, Admin and IoT device

The use-case view identifies the four system actors. Students create and track their own requests and read notifications; staff manage the queue and request workflow; administrators manage users, categories and device keys; and IoT devices post authenticated telemetry that can trigger SYSTEM requests.

2.1 Architecture

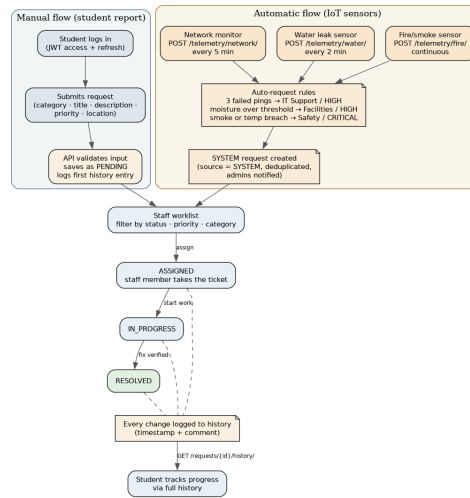


System architecture: three layers from simulated devices and browser clients down through the Django REST API to the database

The system has four cooperating services:

- **Web frontend** — a React single-page application. It holds no business logic beyond presentation; every action goes through the API.
- **Django REST API** — the core. Four apps isolate concerns: **accounts** (users, roles, JWT), **requests** (categories, requests, history, notifications), **telemetry** (device keys, readings, auto-request rules) and **dashboard** (aggregated counters).
- **Telemetry ingest** — technically part of the API but logically separate: it authenticates *devices*, not users, using per-device keys.
- **Simulated sensors** — three ESP32 programs running in Wokwi, each posting readings on its own schedule over plain HTTP.

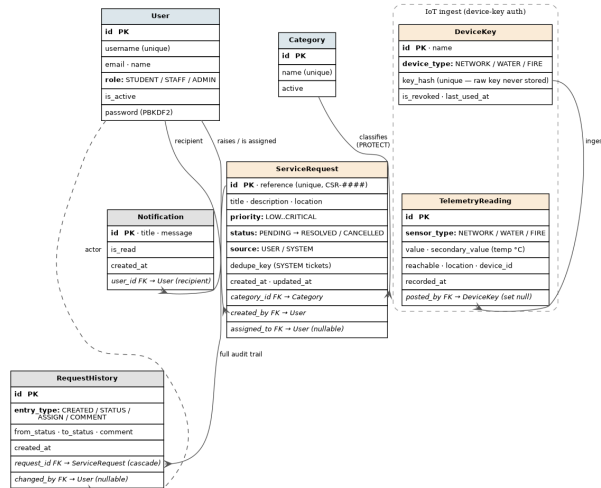
2.2 Data flow



Data flow from a student report or sensor reading to a resolved request

Two paths feed one queue. A student submits a request through the frontend; the API validates it, stamps it **PENDING**, writes the first history entry and notifies staff. A sensor posts a reading; the ingest endpoint authenticates the device key, stores the measurement, and evaluates the rule for that sensor type. If a threshold is breached — and no ticket for that same problem is already open — the API creates a request marked **source = SYSTEM**, which then follows exactly the same workflow as a student’s report. Staff assign, work and resolve; every transition is validated, logged and announced to the affected users.

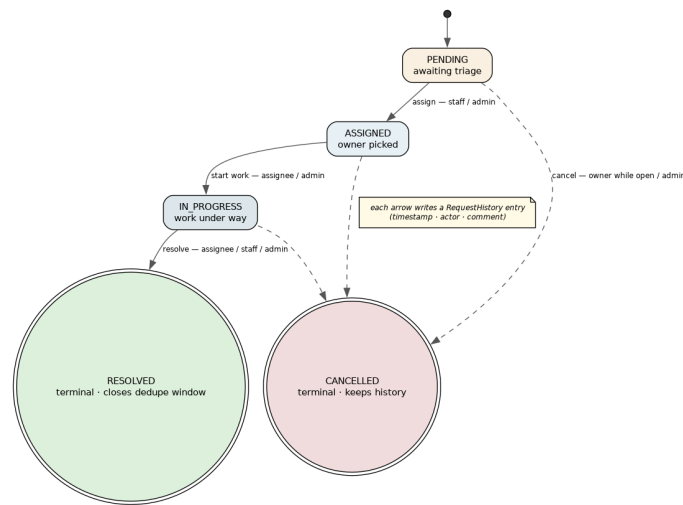
2.3 Data model



Entity relationship diagram for users, requests, history, notifications and telemetry

The data model separates human accounts from device credentials. Categories use **PROTECT** so requests cannot retain an invalid category; request history uses **CASCADE** because it is the request’s audit trail; and telemetry readings use **SET_NULL** for their optional device key reference. **SYSTEM** requests may have no human creator, while assignment remains nullable until staff take ownership.

2.4 Request lifecycle



Request lifecycle state machine with actors and terminal states

Requests move through **PENDING**, **ASSIGNED**, **IN_PROGRESS** and **RESOLVED** in the order enforced by **ALLOWED_TRANSITIONS**. **CANCELLED** is reachable from each open state, and both **RESOLVED** and **CANCELLED** are terminal. Each transition records its actor, timestamp and comment in **RequestHistory**.

2.5 API endpoints

The complete table lives in the repository (**docs/context**, Postman collection); the families are:

Family	Endpoints	Access
Auth	register · login · refresh · logout · me	public / authenticated
Users	list · create · patch · deactivate (admin) · assignable-staff directory for the request drawer	admin / staff
Categories	list · create	authenticated / admin
Requests	CRUD · assign · status · comment · history	owner, staff or admin per object
Notifications	list · mark read	recipient only
Dashboard	role-scoped counters	any authenticated role
Telemetry	/network/ /water/ /fire/ ingest + history	device key (never a user JWT)

2.6 Security design

Security decisions were made up front and enforced in code, not policy documents:

- **JWT everywhere.** Every endpoint requires a bearer token except registration, login and refresh. Refresh tokens rotate and are blacklisted on logout.
- **Roles:** STUDENT, STAFF, ADMIN. Registration always creates a STUDENT account *server-side* — a client cannot elevate itself by posting `"role": "ADMIN"`.

- **Object-level checks.** Permission classes verify not just *can this role call this endpoint* but *may this user touch this specific row*. A student who guesses another student's request ID receives a 404, and the test suite proves it.
- **Device keys are not passwords.** Each sensor gets its own random key; only its SHA-256 hash is stored, keys are bound to one sensor type each, and any key can be revoked independently. A leaked key never exposes user accounts, and a user JWT is worthless at the telemetry endpoints.
- **Passwords** follow Django's validators (length, complexity, common-password checks) and are stored with PBKDF2.

2.7 IoT auto-request rules

Sensor	Cadence	Rule	Request raised
Network monitor	5 min	3 consecutive failed gateway pings	IT Support · HIGH
Water leak	2 min	moisture above 60% (configurable)	Facilities · HIGH
Fire/smoke	10 s	smoke ≥ 40 or temperature ≥ 50 °C	Safety · CRITICAL

Thresholds come from environment variables rather than code, and open SYSTEM tickets are deduplicated per sensor and location: a sensor that keeps tripping raises one ticket, not one hundred. A new ticket appears only after the previous one is resolved or cancelled.

3. Implementation

3.1 Stack and layout

Backend: Python 3 / Django 6 / Django REST Framework with SimpleJWT. Frontend: React 19 (JavaScript), Vite, Tailwind CSS v4, Recharts. Devices: C++ Arduino sketches simulated in Wokwi. Everything lives in one repository:

```

backend/    Django project + apps/{accounts,requests,telemetry,dashboard,core}
frontend/   React SPA (login, dashboard shell, request workflow, charts)
iot/        three Wokwi projects + automated scenario tests
postman/    collection covering every endpoint
docs/       diagrams, test plan, performance results, this report
tools/      performance probe + Playwright walkthrough

```

3.2 Engineering decisions worth defending

One service layer for the workflow. Status changes, assignment and cancellation go through functions in `apps/requests/services.py`. The REST endpoints, the seed command and the telemetry rules all call the same functions, so transition rules, history entries and notifications cannot drift apart depending on who triggered them.

Deduplication at the source. Each SYSTEM request carries a `dedupe_key` (e.g. `water:water-01`). Before raising a ticket the service checks for an open request with the same key. This single field prevents the most likely production failure mode — a stuck sensor flooding the queue — without needing a scheduler or cleanup job.

Hashed device keys, shown once. `seed_demo` prints each raw key exactly once and stores only the SHA-256 hash. Authentication hashes the presented key and looks it up; there is nothing reversible in the database.

Error handling as contract. Validation errors return structured field-level messages; workflow violations return readable explanations (“a request cannot move from PENDING to RESOLVED”); authentication failures return 401 with the correct challenge header; object access violations return 404 so the existence of other rows is never revealed. The frontend surfaces these messages instead of swallowing them.

Configuration by environment. Secret key, CORS origins, token lifetimes, thresholds and even the database engine all come from environment variables with sane defaults, so the same codebase runs on a laptop with SQLite and on a server with MySQL.

3.3 Version control practice

The repository is the audit trail for the build: small commits, descriptive messages, feature work merged directly to `main` and kept runnable. Generated artefacts (virtualenvs, build output, node modules, the SQLite file) are excluded by a curated `.gitignore`; environment templates ship as `.env.example`.

4. Testing and evaluation

4.1 Test plan and unit tests

The full plan is documented in `docs/testing/test-plan.md`; the headline numbers:

Suite	Tests	Covers
Accounts	16	registration forcing STUDENT, password hashing, token lifecycle, admin user management, staff directory scoping
Requests	22	visibility scoping, workflow transitions, history, comments, editing/cancellation rules
Telemetry	19	device-key auth (missing/unknown/revoked/wrong-type), all three auto-request rules, dedupe
Dashboard	4	role-scoped counters, notification privacy
Total	61	all passing under <code>python manage.py test</code>

Every security claim made in section 2.4 has at least one negative test trying to break it. Two examples: `test_student_cannot_read_another_students_request_by_id` and `test_user_jwt_is_not_enough_for_telemetry`.

The IoT firmware is tested two ways. Structural validation (`iot/test/validate_sketches.py`) runs anywhere with no credentials and catches wrong endpoints, missing files and — importantly — any accidentally committed device key. Wokwi automation scenarios (`scenario.yaml`) then drive each simulation: turning the water potentiometer to 90%, heating the DHT22 to 60 °C, letting the network monitor complete a reporting cycle, and asserting the expected serial output.

Finally, a Playwright script walks the real frontend through all three roles, asserting content at every step; the screenshots below are captured by that same script, so the evidence in this report regenerates itself whenever the UI changes.

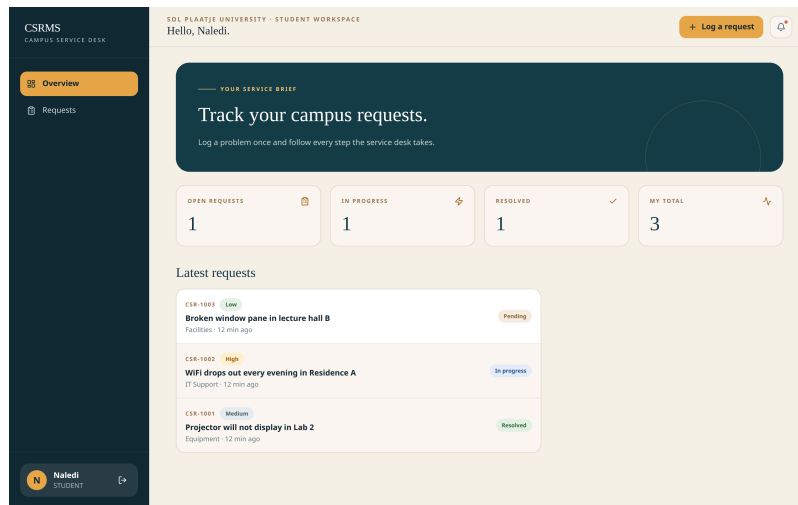
4.2 Performance observations

Using `tools/perf_check.py` (50 timed requests per endpoint against a seeded local server):

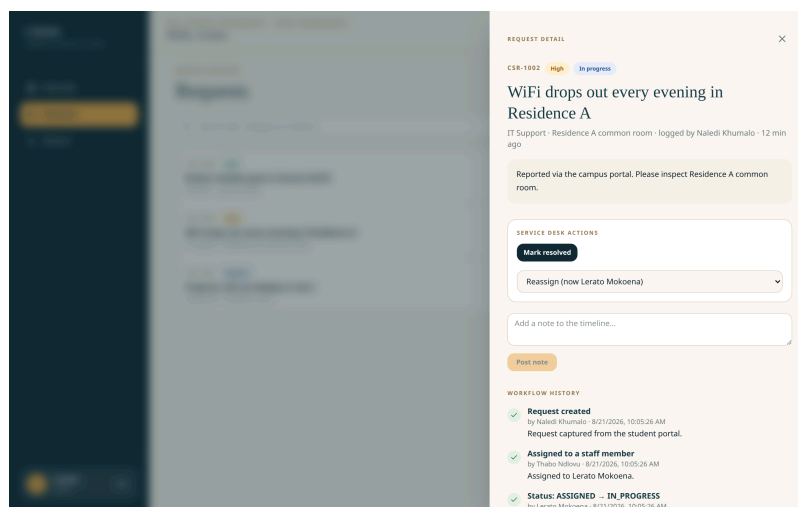
Endpoint	Mean	p95	Max
<code>/api/requests/</code>	4.8 ms	7.4 ms	8.4 ms
<code>/api/dashboard/</code>	4.7 ms	5.9 ms	6.6 ms
<code>/api/notifications/</code>	3.7 ms	5.0 ms	7.5 ms
<code>/api/categories/</code>	4.1 ms	5.9 ms	9.5 ms
<code>/api/users/</code>	4.6 ms	6.5 ms	8.1 ms

Three findings matter more than the absolute numbers. First, JWT verification is local and cryptographic — it never queries the database — so per-request authorisation cost stays flat as load grows. Second, list endpoints are paginated, so response cost is bounded by page size rather than table size; the queue can grow without slowing the app. Third, the known scaling ceiling is SQLite’s serialised writes, which is precisely why the shipped settings switch to MySQL via environment variables for deployment. Device-key lookups are indexed by hash, so telemetry ingest cost does not grow with fleet size.

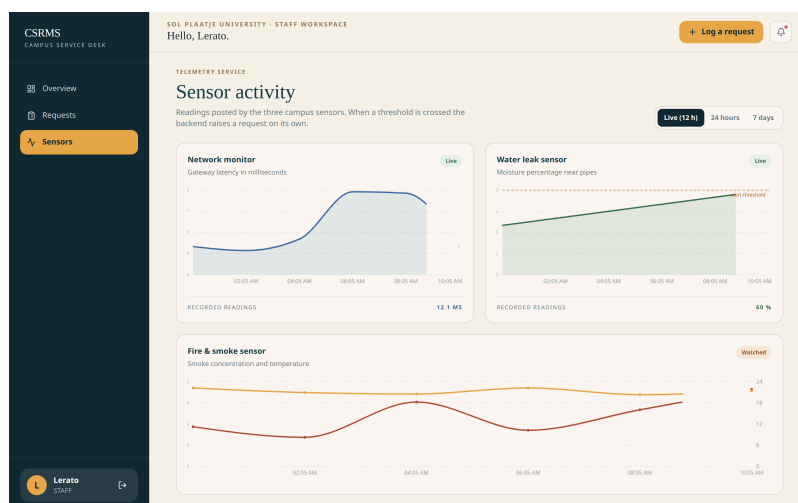
4.3 Evidence



Student dashboard: personal request counters and status breakdown



Staff view of a request with the full workflow history timeline



Sensor activity charts built from twelve hours of stored telemetry

Further captures (registration, notifications, admin user management, the new-request modal) are committed under `docs/screenshots/` and referenced in the demo script. A short recorded walkthrough of the full

workflow — student report, staff triage, simulated sensors raising a SYSTEM ticket — is committed as `docs/demo/assets/csrmc-demo.mp4` and embedded in the slide deck.

5. Results and discussion

Against the objectives set in section 1.3:

1. **REST API** — delivered in full: 24 endpoints across seven families, all documented, exercised by the Postman collection and covered by tests.
2. **Web application** — delivered: role-aware navigation, request workflow with drawer detail and history, filtering, notifications, charts, admin user and category management.
3. **Simulated IoT** — delivered: three devices post on their own schedules and raise tickets through the same workflow as humans, with dedupe keeping the queue honest.
4. **Automated tests** — delivered: 61 API/service tests plus firmware scenarios plus a browser-level smoke test.
5. **Documentation** — delivered: README quick-start, IoT guide, test plan, performance results, diagrams and this report.

The most instructive result is architectural: because SYSTEM requests reuse the exact workflow of manual ones, features like history, notifications and dashboards needed zero extra code to cover sensor-originated incidents. The decision to funnel every state change through one service layer paid for itself repeatedly during development — when we added notification behaviour late in the build, every path (API, seed, telemetry) gained it in one commit.

The evaluation also surfaced genuine limits, recorded honestly in the next section rather than hidden.

6. Limitations and future improvements

Honest limitations of what we built:

- Sensors are simulated. The firmware logic is real, but real hardware would add power management, retry/backoff behaviour and calibration concerns we have not faced.
- Notifications are pull-only (seen in-app when logged in). There is no email or push delivery.
- Performance was measured on a laptop-scale dataset; concurrency behaviour under simultaneous writes was analysed rather than load-tested.
- The fire/smoke rule evaluates instantaneous readings; a sustained-condition window (e.g. N readings above threshold) would be more robust against false positives.

Future work, in priority order:

1. Email/push notification delivery with digest options for staff.
2. SLA tracking: automatic escalation when HIGH/CRITICAL tickets age past a target.
3. Real ESP32 firmware on cheap hardware (the sketches need only WiFi credentials and an endpoint URL changed).
4. A mobile-friendly PWA wrapper so students can report from their phone's home screen.
5. MySQL deployment behind a reverse proxy with HTTPS, plus container images for reproducible hosting.
6. Analytics: resolution-time trends per building/category to inform maintenance budgets.

7. Conclusion

CSRMS demonstrates that a small, disciplined team can deliver the full arc of modern service-system development in one module: a secure REST API, a role-aware web client, automated IoT monitoring, a meaningful automated test suite and documentation a stranger can follow. The system solves the stated campus problem — reports get logged, work gets tracked, sensors speak for the infrastructure — and does it with the same architectural habits (service orientation, least privilege, audit trails, configuration as environment) that scale to real deployments.

Appendix A — Running the system

```
# API
cd backend && python -m venv ../venv && source ../venv/bin/activate
pip install -r requirements.txt && cp .env.example .env
python manage.py migrate && python manage.py seed_demo && python manage.py runserver

# Web app
cd frontend && npm install && cp .env.example .env && npm run dev # :3000

# Tests
cd backend && python manage.py test # 61 tests
cd iot && npm run check # firmware structural checks
node tools/screenshots.mjs # end-to-end browser walkthrough
```

Demo accounts (after `seed_demo`): `admin` / `lerato` / `naledi`, password `Campus#2026`.